VT Dining Project

Requirements Analysis

Group 3 · CS 3704 · Spring 2026

Jaren G · Kyler O · Jai S · Akash P · Justin B · Karthik Y

https://github.com/Fyrlex/cs3704-Group-3-Project

1. Fully Dressed Use Cases

1.1 Kyler O — Backend: User Management & Change Detection

Use Case UC1: User Registration

Primary Actor: New User (A Student)

Stakeholders & Interests: Student: wants a fast and easy account creation experience

System: must store credentials securely and efficiently.

IT Department: needs to comply with VT policy.

Preconditions: The user is not logged in and navigates to the registration page. The application is online and functional.

Success Guarantee: A new user account is created, credentials are stored securely, and the user is redirected to their new profile.

Main Success Scenario: The user navigates to the Register page. The user enters their VT email, username, and password. The system validates that the email ends in @vt.edu. The system hashes the password and stores credentials. The system then sends a verification email. The user clicks the verification link. Finally, the account is activated and the user is logged in.

Extensions: Email is not @vt.edu: System displays error and halts. The verification email bounces: System prompts re-entry. Link expires: System offers to resend.

Special Requirements: Password must meet VT security policy (min 8 chars, 1 uppercase, 1 number). Verification email delivered within a few minutes. These are VT policies as well as conventionally accepted standards.

Technology & Data Variation List: The verification link should be valid for 24 hours. Passwords stored using bcrypt.

Use Case UC2: User Login and Authentication

Primary Actor: Registered User

Stakeholders & Interests: Student: wants fast and secure login

System: must authenticate and issue session tokens quickly

Security: must prevent simple brute-force attacks

Preconditions: The user has a verified account. The application is online and functional.

Success Guarantee: The user is authenticated and receives a session token granting access to protected features.

Main Success Scenario: The user navigates to the Login page. The user enters email and password. The system validates credentials. The system then issues a JWT session token. 5. The user is redirected to the home feed.

Extensions: The credentials do not match: The system displays an error.

Special Requirements: Login must complete within 2 seconds. The session tokens expire after 24 hours of inactivity.

Technology & Data Variation List: JWT tokens for session management. HTTPS required for all auth requests.

Use Case UC3: Profile Management

Primary Actor: Authenticated User

Stakeholders & Interests: Student: wants to personalize profile and manage preferences

System: must handle changes accurately

Preconditions: The user is logged in with a verified account.

Success Guarantee: The user's profile is updated and changes are handled immediately.

Main Success Scenario: The user navigates to Profile Settings. The user updates display name, profile picture, or dining preferences. The system will then validate input and save changes. The system confirms success to the user.

Extensions: Uploaded image exceeds 3 MB: The system rejects and prompts the user to upload a smaller file.

Display name contains profanity: The system rejects request.

Special Requirements: Profile pictures must be JPEG or PNG.

Technology & Data Variation List: Profile pictures stored in cloud storage

Use Case UC4: Change Detection for Dining Data

Primary Actor: Backend System (Job)

Stakeholders & Interests: Students: want awareness when dining data changes

System: must reliably detect and log changes in pricing and quality.

Preconditions: Dining data exists. A scheduled job is configured to run periodically.

Success Guarantee: Any detected changes (price, hours, menu) are logged and queued for notification dispatch.

Main Success Scenario: The scheduled job fetches current dining data snapshot. The system compares to previously stored baseline. If changes are detected, the system records a Change Event with a timestamp, entity ID, field, old and new values. The Change Event is pushed to the notification queue. The system updates the baseline snapshot.

Extensions: No changes detected: The system updates last-checked timestamp and exits.

Data source unreachable: The system logs an error and retries after 3 minutes.

Special Requirements:

Technology & Data Variation List: Some sort of diff for comparisons, runs every 5 minutes.

1.2 Jai S — ML & Analytics

Use Case UC5: NLP Review Summarization

Primary Actor: Student (Viewer)

Stakeholders & Interests: Student: wants a quick text summary of many reviews without reading each one; System: must produce accurate and concise summaries.

Preconditions: At least 10 text reviews exist for the selected dining entity. NLP model is deployed.

Success Guarantee: A concise summary paragraph (3–5 sentences) is displayed on the dining entity's page.

Main Success Scenario: 1. Student opens a dining location page. 2. System retrieves the last 10 text reviews. 3. NLP service generates a summary. 4. Summary is displayed in the 'Quick Summary' section.

Extensions: 3a. NLP service unavailable: System shows 'Summary temporarily unavailable' and displays raw reviews. 3b. Reviews too short to summarize: System shows top 3 recent reviews instead.

Special Requirements: Summary generation must complete within 3 seconds. Summary must be readable at a 6th-grade level.

Technology & Data Variation List: LLM-based summarization using a fine-tuned transformer. Summaries cached for 30 minutes.

Use Case UC6: Sentiment Analysis on Reviews

Primary Actor: Analytics Service

Stakeholders & Interests: Students: want to see overall sentiment for a dining location; System: must classify sentiment accurately.

Preconditions: New review is submitted. Sentiment model is deployed.

Success Guarantee: Sentiment label (Positive, Neutral, Negative) and confidence score are stored with the review.

Main Success Scenario: 1. New review is submitted. 2. Sentiment service receives review text. 3. Service classifies sentiment and returns a label and confidence score. 4. Label and score are stored with the review. 5. Aggregate sentiment scores for the entity are updated.

Extensions: 3a. Confidence score below 0.6: System marks sentiment as 'Undetermined' and flags for manual review.

Special Requirements: Sentiment scoring must complete within 1 second of review submission.

Technology & Data Variation List: Fine-tuned BERT classifier. Batch reprocessing available for model updates.

Use Case UC7: Busy Hour Prediction

Primary Actor: Student (Viewer)

Stakeholders & Interests: Student: wants to know the best time to dine to avoid crowds; Dining Hall: accurate predictions support staffing decisions.

Preconditions: At least 90 days of visit data exists for the dining location.

Success Guarantee: A busy-hour prediction chart is displayed showing predicted crowd levels by hour for each day of the week.

Main Success Scenario: 1. Student opens a dining location page. 2. Student selects 'Best Time to Go' tab. 3. System retrieves historical visit data. 4. ML model predicts busyness levels per hour per weekday. 5. Predictions displayed as a color-coded heatmap.

Extensions: 4a. Insufficient historical data: System shows 'Not enough data yet' message.

Special Requirements: Predictions updated at least weekly. Heatmap must use a color-blind friendly palette.

Technology & Data Variation List: Time-series regression model. Data sourced from aggregated check-in events.

Use Case UC8: Chatbot for Dining Questions

Primary Actor: Student

Stakeholders & Interests: Student: wants quick answers about dining options, hours, and menus; System: must provide accurate and helpful responses.

Preconditions: Student is on the platform. Chatbot service is running. Dining data is up-to-date.

Success Guarantee: Student receives a relevant, accurate answer to their dining query within 5 seconds.

Main Success Scenario: 1. Student opens the chatbot interface. 2. Student types a question. 3. Chatbot retrieves relevant data from the dining entity database. 4. Chatbot generates an appropriate response using retrieved data. 5. Student can ask follow-up questions. 6. The process repeats.

Extensions: 3a. Query cannot be matched: Chatbot responds with 'I don't have that information' and suggests browsing listings. 5a. Offensive or off-topic question: Chatbot politely redirects to dining-related topics.

Special Requirements: Chatbot must respond within 5 seconds. Must not store personally identifiable conversation data.

Technology & Data Variation List: RAG (Retrieval-Augmented Generation) with a fine-tuned LM and the dining entities database as context. Implementation of the guardrails framework, like NVIDIA NeMo to prevent misuse.

1.3 Akash P — Backend: Dining Entity & Administration

Use Case UC9: Dining Entity Management

Primary Actor: Administrator

Stakeholders & Interests: Administrator: wants to accurately maintain the catalog of dining locations; Students: expect accurate and current dining location data.

Preconditions: Administrator is logged in with admin privileges.

Success Guarantee: Dining entity is created, updated, or deactivated, and changes are immediately reflected on the platform.

Main Success Scenario: 1. Admin navigates to Admin Panel > Dining Entities. 2. Admin creates a new entity or selects an existing one. 3. Admin fills in or updates fields: name, location, category, hours, contact info. 4. System validates required fields. 5. System saves changes and updates the public listing.

Extensions: 4a. Required fields are missing: System highlights missing fields and prevents saving. 5a. Entity name conflicts with existing record: System prompts admin to confirm or rename.

Special Requirements: Entity names must be unique. Changes to hours must propagate to the notification system within 2 minutes.

Technology & Data Variation List: Dining entity data stored in a relational database. Hours support complex schedule formats (e.g., different hours per day).

Use Case UC10: Menu and Pricing Data Management

Primary Actor: Administrator

Stakeholders & Interests: Administrator: wants to keep menu and pricing data current; Students: rely on accurate menu data to make dining decisions.

Preconditions: Administrator is logged in with admin privileges. Dining entity exists.

Success Guarantee: Menu items and prices are updated and immediately reflected on the dining entity's public page.

Main Success Scenario: 1. Admin navigates to a dining entity's Menu Management section. 2. Admin adds, edits, or removes menu items. 3. Admin sets or updates pricing. 4. System validates pricing. 5. System saves and publishes the updated menu.

Extensions: 4a. Invalid price format: System highlights field and prevents saving. 3a. Admin deletes an item referenced in existing reviews: System warns and requires confirmation.

Special Requirements: Prices must display in USD with two decimal places. Menu updates must be versioned for audit purposes.

Technology & Data Variation List: Pricing data stored with effective dates to support historical lookups.

Use Case UC11: Review Moderation

Primary Actor: Moderator / Administrator

Stakeholders & Interests: Students: expect a safe review environment free of abuse; Platform: must enforce community guidelines to maintain trust.

Preconditions: Reviews have been submitted. Moderator is logged in with moderation privileges.

Success Guarantee: Flagged or violating reviews are reviewed and appropriate action (approved, edited, or removed) is taken.

Main Success Scenario: 1. System automatically flags reviews containing prohibited keywords or low sentiment confidence. 2. Moderator navigates to the Moderation Queue. 3. Moderator reads flagged review and selects action: Approve, Edit, or Remove. 4. System records the moderation action with timestamp and moderator ID. 5. If removed, review is hidden and reviewer is notified.

Extensions: 3a. Moderator is unsure: Can escalate to a senior admin. 5a. Reviewer disputes removal: A dispute workflow is triggered for senior admin review.

Special Requirements: Moderation decisions must be logged for audit. Moderators must not moderate their own reviews.

Technology & Data Variation List: Automated flagging uses keyword blocklist and ML sentiment threshold. All moderation actions are immutable audit records.

Use Case UC12: System Configuration Management

Primary Actor: System Administrator

Stakeholders & Interests: System Admin: needs to adjust platform-wide settings; Development Team: needs config changes applied without redeployment.

Preconditions: Administrator is logged in with system-admin privileges.

Success Guarantee: Configuration changes are saved and applied without requiring a system restart.

Main Success Scenario: 1. Admin navigates to System Configuration panel. 2. Admin adjusts settings such as notification frequency, ML thresholds, or feature flags. 3. System validates values are within allowed ranges. 4. System saves config and propagates changes to relevant services. 5. Admin receives confirmation that changes are live.

Extensions: 3a. Invalid configuration value: System rejects and displays allowed range. 4a. Downstream service fails to acknowledge: System alerts admin and reverts to previous config.

Special Requirements: Config changes must be applied within 30 seconds. All config changes must be logged with actor ID and timestamp.

Technology & Data Variation List: Configuration stored in a distributed key-value store. Feature flags support percentage-based gradual rollout.

1.4 Justin B: Review & Feedback Management

Use Case UC13: Submit Dining Feedback

- **Primary Actor:** Student
- **Stakeholders & Interests:** Student: wants to provide direct feedback to dining halls; Dining Hall Managers: want to receive actionable feedback to improve services.
- **Preconditions:** Student is logged in.
- **Success Guarantee:** Feedback is successfully saved and routed to the appropriate dining hall administration.
- **Main Success Scenario:**
    - Student navigates to the "General Feedback" section.
    - The student selects a dining entity and enters feedback text.
    - The system validates the input and saves the record.
- **Extensions:**
    - 3a. Empty feedback: System prompts the user to enter text before submitting.

Use Case UC14: Submit Dining Review

- **Primary Actor:** Student
- **Stakeholders & Interests:** Student: wants to share their dining experience; Other Students: rely on reviews for dining decisions.
- **Preconditions:** Student is logged in and has not reviewed this specific dining hall for the current visit.

- **Success Guarantee:** Review is published, and the entity's aggregate rating is updated.
- **Main Success Scenario:**
    - The student selects a dining hall and chooses "Write a Review".
    - Students provide a star rating and optional text/photos.
    - System verifies the one-review-per-visit rule.
    - The system saves the review and updates the live feed.
- **Extensions:**
    - 3a. Duplicate review detected: System informs the student they have already submitted a review for this visit.

Use Case UC15: Manage Personal Reviews

- **Primary Actor:** Student
- **Stakeholders & Interests:** Student: wants to correct or delete their own content; System: must ensure users can only modify their own data.
- **Preconditions:** At least one review submitted by the user exists in the system.
- **Success Guarantee:** The review is successfully updated or removed from public view.
- **Main Success Scenario:**
    1. The student navigates to "My Reviews" on their profile.
    2. The student selects the "Edit" or "Delete" option for a specific review.
    3. The system confirms the student is the original author.
    4. The system saves the changes or flags the review as deleted.

Use Case UC16: User Achievement Tracking

- **Primary Actor:** Student
- **Stakeholders & Interests:** Student: wants recognition for community contributions; System: wants to encourage high-quality reviews.
- **Preconditions:** Student has submitted at least one review.
- **Success Guarantee:** Student's "Reviewer Level" or badge status is updated based on their activity.
- **Main Success Scenario:**
    1. The student submits a review.
    2. The system increments the user's review count.
    3. System checks if the user has met a milestone (e.g., 10 reviews).
    4. The system awards a new badge and displays it on the user's profile.

1.5 Jaren G — Review Interface (Discovery/Viewing)

Use Case UC17: Recent Reviews Feed

Primary Actor: Student (Viewer)

Stakeholders & Interests: Students: want to see the most recent reviews from across the platform; System: must surface recent activity quickly.

Preconditions: User is on the home page. At least 5 reviews exist in the system.

Success Guarantee: A paginated feed of the most recent reviews is displayed with reviewer name, dining entity, rating, and excerpt.

Main Success Scenario: 1. User navigates to the home page. 2. System queries the 20 most recent reviews. 3. Feed is rendered with cards showing entity name, rating, review excerpt, and timestamp. 4. User can click a card to view the full review. 5. User can paginate or infinite-scroll to load more.

Extensions: 2a. No reviews exist: System displays an empty state message with a prompt to write the first review.

Special Requirements: Feed must render in under 1 second. Cards must be keyboard-navigable.

Technology & Data Variation List: Feed is server-side paginated with a cursor. Images are lazy-loaded.

Use Case UC18: Review Filtering and Search

Primary Actor: Student (Viewer)

Stakeholders & Interests: Students: want to find reviews relevant to specific dining halls, food types, or rating ranges.

Preconditions: User is on the reviews browse page.

Success Guarantee: Filtered or searched results are displayed correctly and update as filters are adjusted.

Main Success Scenario: 1. User enters a search term or selects filter criteria (dining hall, category, rating range, date range). 2. System sends a filtered query to the backend. 3. Matching reviews are returned and displayed. 4. User can clear filters to return to the full list.

Extensions: 3a. No reviews match: System displays 'No results found' with suggested adjustments. 1a. Search term is blank: System shows all results.

Special Requirements: Search results must update within 500ms of filter change. Filters must be bookmarkable via URL parameters.

Technology & Data Variation List: Full-text search powered by an inverted index. Filters applied server-side.

Use Case UC19: Detailed Review Pages

Primary Actor: Student (Viewer)

Stakeholders & Interests: Students: want to read the full details of a review including all ratings, photos, and responses.

Preconditions: Review exists and is publicly visible.

Success Guarantee: Full review details are displayed including all sub-ratings, text, photos, reviewer info, and dining entity link.

Main Success Scenario: 1. User clicks a review card or follows a direct link. 2. System fetches full review data including sub-ratings, photos, and metadata. 3. Page renders with all details. 4. User can navigate to the reviewer's profile or the dining entity page.

Extensions: 2a. Review has been deleted: System displays a '404 - Review Not Found' page.

Special Requirements: Page must be shareable via direct URL. Photos must load progressively.

Technology & Data Variation List: Review photos stored in CDN. Page must support Open Graph metadata for social sharing.

Use Case UC20: Review Trend Visualization

Primary Actor: Student (Viewer)

Stakeholders & Interests: Students: want to see how a dining entity's ratings have changed over time to make informed dining decisions.

Preconditions: Dining entity has at least 10 reviews over at least 2 weeks.

Success Guarantee: An interactive chart displaying rating trends over time is shown on the dining entity page.

Main Success Scenario: 1. The user navigates to a dining entity's page. 2. The user selects the 'Trends' tab. 3. The system retrieves pre-aggregated trend data. 4. A line chart is rendered showing average rating per week over the past 6 months. 5. Users can hover over data points to see review counts.

Extensions: 3a. Insufficient data: System shows 'Not enough data yet' message.

Special Requirements: UI must render in under 1 second. The chart must use an accessible color palette with descriptive axis labels.

Technology & Data Variation List: Trend data sourced from the pre-aggregated trend cache. Chart rendered with a JS charting library (e.g., Chart.js).

1.6 Karthik Y — Backend: Notifications

Use Case UC21: Event-Driven Notifications

Primary Actor: Notification Service

Stakeholders & Interests: Students: want to be instantly informed of relevant changes; System: must deliver notifications reliably and promptly.

Preconditions: Student has an active account and has opted into notifications. A Change Event has been queued.

Success Guarantee: Notification is composed and delivered to the student via their preferred channel within 30 seconds of the event.

Main Success Scenario: 1. Change Event is received from the notification queue. 2. Notification service retrieves the list of subscribed students for the affected entity. 3. For each subscriber, service composes a personalized notification. 4. Service dispatches via the student's preferred channel. 5. Delivery status is logged.

Extensions: 4a. Delivery fails: System retries up to 3 times with exponential backoff. 2a. No subscribers exist: Service logs and discards the event.

Special Requirements: Notifications must be delivered within 30 seconds of event queuing. Must support bursts of 1000+ notifications per minute.

Technology & Data Variation List: Message queue: Apache Kafka or RabbitMQ. Push notifications via FCM; email via SendGrid.

Use Case UC22: Trend-Based Alerts

Primary Actor: Analytics Service / Notification Service

Stakeholders & Interests: Students: want to know when a dining hall's quality has notably improved or declined; System: must identify meaningful trends and avoid alert fatigue.

Preconditions: Trend data has been computed for a dining entity. Student is subscribed to trend alerts.

Success Guarantee: A trend alert is sent when average rating changes by more than 0.5 stars over a rolling 7-day window.

Main Success Scenario: 1. Analytics service computes latest trend data. 2. Service compares latest 7-day average to prior 7-day average. 3. If change exceeds threshold (0.5 stars), a Trend Alert Event is generated. 4. Notification service picks up the event and dispatches alerts to subscribed students.

Extensions: 3a. Change is below threshold: No alert is generated. 3b. Insufficient data for meaningful comparison: Alert is suppressed.

Special Requirements: Trend alerts must not be sent more than once per entity per 24-hour period to prevent alert fatigue.

Technology & Data Variation List: Trend comparison uses pre-aggregated rolling averages from the trend cache.

Use Case UC23: Multi-Channel Notification Support

Primary Actor: Notification Service

Stakeholders & Interests: Students: want to receive notifications via their preferred channel; System: must support multiple delivery channels without duplicating messages.

Preconditions: Student has configured notification preferences with at least one channel enabled.

Success Guarantee: Notification is delivered through all channels the student has enabled, without duplication.

Main Success Scenario: 1. Notification service retrieves student's channel preferences (in-app, email, push). 2. Service dispatches notification to each enabled channel concurrently. 3. Each channel logs delivery success or failure independently. 4. Student receives notification on all enabled channels.

Extensions: 2a. One channel fails: Service logs failure but continues delivering to other channels. 1a. All channels disabled: Service logs and discards.

Special Requirements: Each channel delivery must be independent — failure on one must not affect others.

Technology & Data Variation List: Channels: in-app (WebSocket), email (SendGrid), mobile push (FCM). All channels use the same message payload.

Use Case UC24: Notification Preference Management

Primary Actor: Authenticated User

Stakeholders & Interests: Students: want control over what types of notifications they receive and via which channels.

Preconditions: User is logged in. Notification settings page is accessible.

Success Guarantee: User's notification preferences are updated and immediately applied to future dispatch.

Main Success Scenario: 1. User navigates to Settings > Notifications. 2. User toggles notification types (price changes, quality alerts, new reviews) on/off. 3. User selects preferred delivery channels per notification type. 4. System saves preferences. 5. System confirms changes are saved.

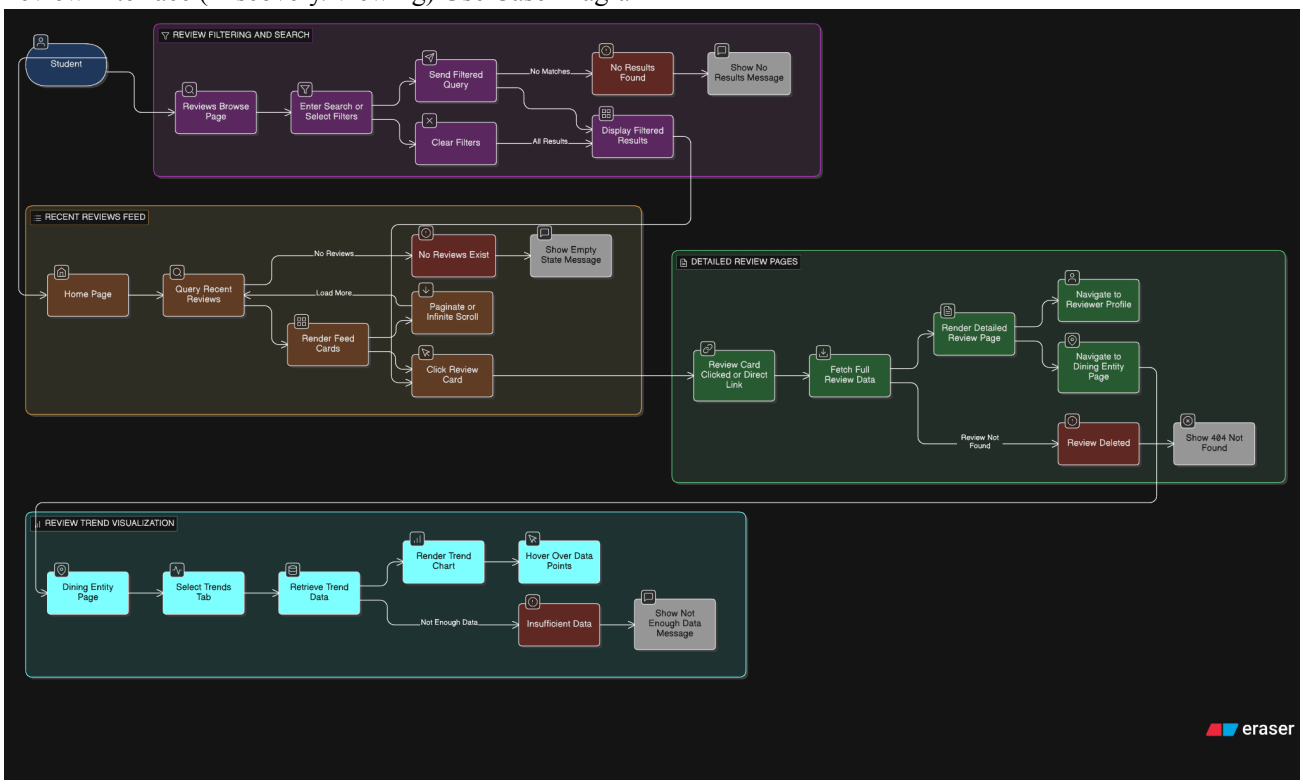
Extensions: 4a. Save fails due to network error: System displays error and retains previous settings.

Special Requirements: Changes must take effect within 60 seconds. UI must clearly distinguish between notification types.

Technology & Data Variation List: Preferences stored per-user in the database. Preference schema versioned to support future notification types.

2. Use Case Diagrams

Review Interface (Discovery/Viewing) Use Case Diagram



3. Conceptual Class Diagrams

Put class diagrams here

4. Supplementary Specifications

NFR = Non-Functional Requirement

4.1 Responsive Performance — Jaren G

NFR-1: All user-facing UI screens must render and become interactive within 1 second under normal campus WiFi conditions. This applies to the reviews feed, dining entity pages, and trend visualization charts. Server responses must be under 300ms for 95th-percentile requests.

4.2 Security — Kyler O

NFR-2: All authentication credentials must be transmitted over HTTPS. Passwords must be stored using bcrypt with a minimum cost factor of 12. Session tokens must expire after 24 hours of inactivity. Login endpoints must implement rate limiting with account lockout after 5 failed attempts.

4.3 ML Model Accuracy — Jai S

NFR-3: The sentiment analysis model must achieve a minimum F1 score of 0.80 on the held-out test set before deployment. The NLP summarization service must be rated as coherent and accurate by at least 85% of test users. The busy-hour prediction model must achieve a mean absolute error of less than 15 minutes on predicted peak times.

4.4 Data Integrity & Auditability — Akash P

NFR-4: All administrative actions (entity creation, menu updates, moderation decisions, system config changes) must be recorded in an immutable audit log with actor identity and timestamp. Audit logs must be retained for at least 12 months. Menu pricing changes must be versioned so historical pricing can be retrieved.

4.5 Accessibility — Justin B

NFR-5: The platform must meet WCAG 2.1 Level AA accessibility standards. This includes a minimum color contrast ratio of 4.5:1 for normal text, full keyboard navigability for all interactive elements, compatibility with major screen readers (NVDA, VoiceOver), and descriptive alt text for all non-text content. The application must function correctly on mobile viewports down to 375px width.

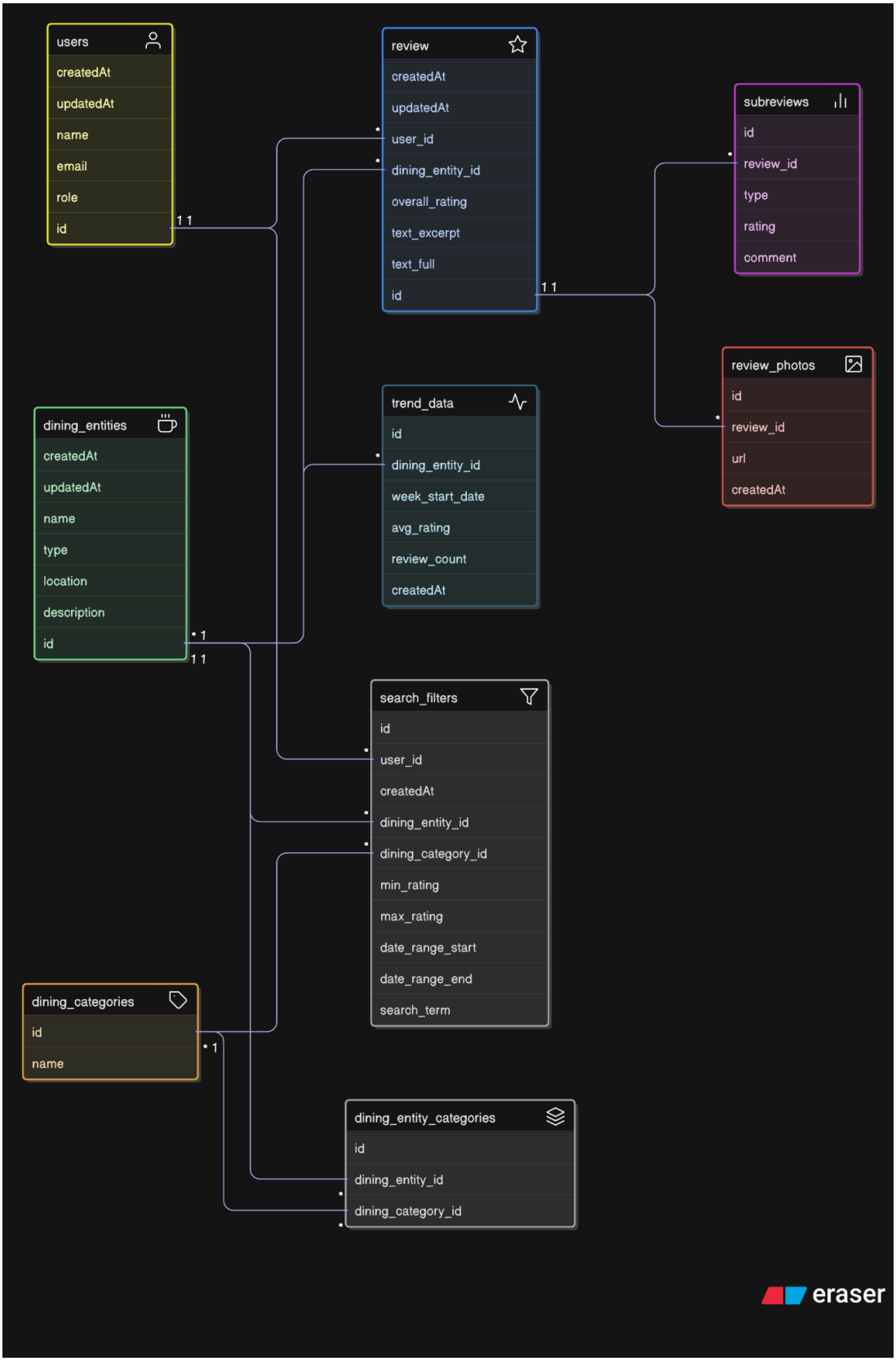
4.6 Notification Reliability — Karthik Y

NFR-6: The notification system must achieve a delivery success rate of at least 99% for in-app and email channels under normal operating conditions. Push notifications must be delivered within 30 seconds of the triggering event. If one notification channel is unavailable, notifications must still be delivered via remaining enabled channels. No more than 5 trend-based alerts per user per day may be dispatched.

Class diagrams: A

Class diagrams: B

Review Interface (Discovery/Viewing)



Usecasepdf

— End of Requirements Analysis Document —